

Нишки и конкурентност

1. Какво е нишка (Thread)

Нишката е по-малка единица за изпълнение от процеса. Докато процесът е програма в изпълнение с отделна памет, нишката споделя паметта с други нишки от същия процес.

Разлика между процес и нишка:

- **Процес** - има своя памет, файлове, ресурси
- **Нишка** - споделя паметта и ресурсите с други нишки (по-лека единица)

Аналогия: Процес = фабрика, Нишка = работник в фабриката

Пример 1: Web браузър (Chrome, Firefox, Edge)

Процес: Един таб в Chrome често е отделен процес → има собствена памет, за да не „счупи“ другите табовете при срив.

Нишки в този процес:

- Нишка за UI (използва се за визуализиране на прозореца)
- Нишка за рендиране (HTML/CSS/JS)
- Нишка за network заявки (зареждане на страници)

- Нишка за garbage collection (в случая на JS двигатели като V8)

Пример 2: Музикален плейър (Spotify, VLC, Winamp)

Процес: Приложението, което стартира (например vlc.exe)

Нишки:

- Нишка за възпроизвеждане на звук
- Нишка за UI (бутон Play/Pause, прозорци и менюта)
- Нишка за четене на файлове от диска
- Нишка за декодиране на аудио формати (mp3, flac)

Ключово: Така музиката не прекъсва, дори ако потребителят отваря менюта или плейърът зарежда следваща песен.

2. Мултинишки процеси

ПРОЦЕС (обща памет)

Нишка 1

Нишка 2

Нишка 3

Всички нишки споделят паметта и ресурсите на процеса!

3. Конкурентност и паралелизъм

Конкурентност

- Един процесор
- Задачите се редуват
- Изглежда едновременно

Паралелизъм

- Много процесори/ядра
- Задачите работят реално едновременно
- Наистина едновременно

Аналогия:

- **Конкурентност** - един сервитьор обслужва 10 маси (бързо се върти)
- **Паралелизъм** - 10 сервитьори обслужват 10 маси едновременно

4. Предимства и проблеми

Предимства:

- ☒ Използва всички ядра на процесора
- ☒ Програмата не замръзва
- ☒ Няколко задачи едновременно

Пример: Web браузър с нишки

Когато отвориш уеб страница, браузърът използва множество нишки:

- **Нишка 1:** Зарежда HTML текста от сървър

- **Нишка 2:** Зарежда CSS стиловете
- **Нишка 3:** Изпълнява JavaScript кода
- **Нишка 4:** Сваля снимки и изображения
- **Резултат:** Страницата се зарежда много по-бързо, защото всичко се случва едновременно!

Race Condition (Състезание):

Когато две нишки опитват да променят една и съща памет едновременно.

Пример: Сметка с 100 лева. Две нишки опитват да добавят пари едновременно - може да загубят промени.

Deadlock (Мъртва хватка):

Две нишки чакат една друга и никоя не може да продължи. Програмата замръзва.

5. Синхронизация

Синхронизацията е координацията на нишките да не се пречат една на друга.

Mutex

- Като катинар
- Само една нишка може да влезе

- Другите чакат

Semaphore

- Брояч
- Позволява определен брой нишки
- Например 3 нишки едновременно

Lock

- Блокира достъп
- Критична секция
- Защишава данни

Пример с Mutex за банкова сметка:

Нишка 1:

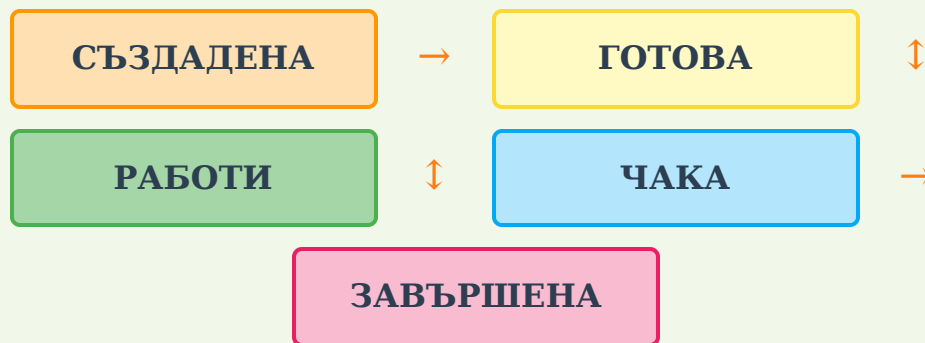
Lock → Чете 100 → Добавя 50 → Записва 150 → Unlock

Нишка 2:

Lock (чака!) → Чете 150 → Добавя 30 → Записва 180 → Unlock

Резултат: 180 лева ✓

6. Състояния на нишката



7. Заключение

Нишките позволяват на програмите да използват всички ядра на процесора и да не замръзват.

Запомни:

- **Нишка** - по-малка единица от процеса
- **Конкурентност** - много задачи изглеждат едновременно
- **Race Condition** - нишки се пречат
- **Deadlock** - нишки се чакат
- **Синхронизация** - Mutex, Semaphore, Lock
- **Състояния** - Създадена, Готова, Работи, Чака, Завършена